

《Codex 完整入门与实用指南》

面向大学 IT / Programming 学生的学习笔记、操作手册与 Prompt Library

阅读建议：先读第 1-6 节建立安全学习方法，再用第 7 和第 14 节复制 prompt 实战。写 assignment 时优先使用检查、解释、debug 和最小修改流程。

1. Codex 是什么

Codex 是 OpenAI 的 AI coding agent。你可以把它理解成一个会阅读项目、解释代码、提出修改、编辑文件、运行命令和检查结果的编程学习搭档。它不是单纯的“代码补全”，而是能围绕一个任务持续工作。

一句话理解：ChatGPT 更像会讲解的老师；普通补全工具更像打字助手；Codex 更像可以进入项目现场、读文件、跑测试、提交修改建议的编程搭档。

概念	初学者解释
Codex 和普通 ChatGPT 的区别	ChatGPT 通常根据你粘贴的内容回答；Codex 可以在项目文件夹中阅读多个文件、执行命令、给出 diff，并围绕 bug/测试/PR 完成多步任务。
Codex 和 GitHub Copilot 的区别	Copilot 强在 IDE 中实时补全和局部建议；Codex 更偏 agent workflow，能接收一个目标后自己探索项目、修改文件、运行测试和总结结果。
Codex 和普通代码补全工具的区别	普通补全主要预测下一行；Codex 会理解需求、项目结构、错误输出和测试结果。
什么是 coding agent	一种能使用工具的 AI：它不只是回答，还能读文件、编辑、运行命令、检查输出，并在得到新信息后继续调整。
适合哪些人	IT/programming 学生、刚接触项目的初学者、需要 debug 的开发者、团队项目成员、想练习 Git/GitHub 和 code review 的学习者。

重要提醒：本文档把 Codex 当作学习导师和工程助手，而不是代写机器。最好的使用方式是让它解释、检查、定位、提出小修改，然后你自己理解、验证并负责最终提交。

2. Codex 能帮我做什么

下面这张表可以当作日常使用速查。每次使用 Codex 前，先想清楚：我是在学习、检查、debug，还是准备提交？不同目标需要不同 prompt。

用途	具体能做什么	适合场景	示例 prompt	注意事项
解释代码	把陌生代码按变量、函数、流程、输入输出、边界情况讲清楚。	看老师示例、同学项目、旧代码、报错附近代码。	请用初学者能懂的中文解释这段代码：每个函数做什么、数据如何流动、我需要掌握哪些语法点。	不要只看解释就提交；要自己复述逻辑，并能手动改一处小功能。
写代码	根据需求生成函数、组件、脚本或测试，但应先讨论方案。	练习小功能、搭建骨架、补全重复模板。	先不要写完整答案。请根据我的 assignment 要求给出实现思路、伪代码和我应该自己完成的步骤。	assignment 中避免让它直接完成全部核心答案；保留自己的设计和实现过程。
修改代码	做小范围重构、修复变量名、拆函数、清理重复逻辑。	代码能跑但结构混乱、重复很多、老师要求可读性。	请只修改 input validation 相关代码，不要重写其他函数。输出修改原因和 diff 摘要。	限制范围，先看 diff，再运行测试。
Debug	阅读错误现象、复现步骤、日志和代码，定位可能原因。	程序结果不对、某个按钮无效、测试失败。	这是我的预期输出、实际输出和相关代码。请先列出 3 个可能原因，再告诉我如何验证。	不要接受第一种猜测；要求它给验证方法。
查找 compile error	解释编译器错误、找出语法或类型问题、建议最小修复。	C++/Java/TypeScript 编译失败。	请解释这个 compile error 的真正含义，指出最可能的源代码位置，并给 beginner-friendly 的最小修复。	贴完整错误，不只贴最后一行。
运行测试	运行已有 test、解释失败、生成更小复现。	提交前检查、CI failed、改 bug 后验证。	请运行项目现有测试。如果失败，先总结失败原因，不要马上大改。	本地环境不同会导致结果不同；记录命令和输出。
阅读整个项目结构	浏览目录、入口文件、模块依赖和运行方式。	接手团队项目、GitHub repo、老师 starter code。	请阅读项目结构，画出主要文件作用、程序入口、数据流和我应该先看哪些文件。	先让它只分析，再决定是否修改。
检查 assignment rubric	把 rubric 转成检查清单，逐项对照代码。	提交前、担心漏要求、想提高分数。	请根据 rubric 检查我的项目，每项标注：已满足/部分满足/未满足/需要我确认。	rubric 不是绝对判分器；老师解释优先。

用途	具体能做什么	适合场景	示例 prompt	注意事项
生成 README	生成安装、运行、功能、文件结构、已知限制。	课程项目、GitHub 提交、团队交付。	请根据这个项目生成 README，语言保持学生风格，不要夸大功能。	README 必须和真实功能一致。
写 comments	给复杂函数加少量解释性注释。	老师要求 comments、代码逻辑不直观。	请只在必要位置添加 beginner-level comments，不要每行都注释。	过度注释会显得不自然，优先改善命名。
生成 test cases	覆盖正常、边界、错误输入和回归测试。	函数完成后、debug 后、准备提交。	请为这个函数设计测试用例表：输入、预期输出、覆盖目的，不要直接写完整测试代码。	测试也可能错；先人工验证预期。
检查安全问题	检查 hardcoded secret、路径遍历、XSS、SQL injection、权限问题。	web assignment、cybersecurity lab、公开 repo。	请做防御性安全检查：只指出风险、原因、修复建议，不提供攻击利用步骤。	不要让它帮助未授权攻击或绕过系统。
优化代码结构	拆分函数、降低重复、改善命名、分离 UI 和逻辑。	项目变大、团队协作、老师要求 maintainability。	请提出 3 个小型重构建议，每个建议说明收益、风险和是否超出初学者范围。	不要追求企业级架构；保持课程范围。
准备老师可能提问	根据代码生成 viva/demo 问题和参考回答。	演示、口头答辩、代码检查。	请扮演老师，根据我的代码问 10 个问题，覆盖逻辑、边界情况、我为什么这样设计。	回答要基于你真正理解的代码。
辅助 cybersecurity lab	解释命令、输出、概念、报告语言和防御建议。	合法课程实验、靶场、CTF training。	这是我在授权 lab 中的命令输出。请解释每一段含义、风险等级和报告写法，不要给非法攻击步骤。	只在授权环境使用；避免真实目标、凭证窃取和持久化。
辅助 GitHub pull request	总结改动、review 风险、检查测试、写 PR 描述。	团队项目、开源练习、作业版本管理。	请 review 这个 PR，优先找 bug、缺失测试和不符合 requirement 的地方。	不要把 AI review 当最终审批；团队规范优先。
团队项目 code review	按文件、责任人、功能点提出可执行反馈。	多人项目 merge 前。	请像 code reviewer 一样检查这次改动，按 severity 排序，并给出具体的文件位置。	反馈要尊重队友，只讨论代码和需求。

3. Codex 的主要使用方式

3.1 在 ChatGPT 里使用 Codex

- 适合：概念学习、解释代码片段、分析 assignment requirement、生成 prompt、准备答辩问题。
- 上传代码/截图/文件：把相关文件、错误截图、terminal output、rubric 放在同一轮对话里，并说明课程语言和你的水平。
- 让它解释代码：要求按“整体目的 -> 函数 -> 关键变量 -> 执行流程 -> 常见错误”解释。
- 让它检查 assignment：先提供 requirement/rubric，再提供自己的代码，让它逐项检查，不要直接重写。
- 优点：上手快、适合问问题；限制：如果没有完整项目，它只能根据你提供的材料判断。

我是一名大学 IT 初学者。请先阅读 assignment requirement 和 rubric，只做检查清单，不要写代码。然后告诉我需要提供哪些文件才能进一步检查。

3.2 在 VS Code 中使用 Codex

- 适合：边写代码边解释、修复当前文件、检查多个文件关系、review diff。
- 读取当前项目：在项目根目录打开 VS Code，让 Codex 从 workspace 读取文件结构；第一次任务先问“请总结项目结构”。
- 只检查当前文件：明确写“只阅读/只修改当前文件”，并让它先列出计划。
- 理解多文件关系：提供入口文件、相关模块名、报错信息，让它追踪 import/include、function call、组件关系。
- 不要乱改：prompt 中写清修改范围、禁止重写、禁止引入新库、保持 beginner-level style。
- 查看和接受修改：逐个看 diff，确认改了哪些文件、每个修改为什么需要、是否能通过运行/测试。

请只检查当前文件是否有明显 bug 和 readability 问题。先列出发现，不要修改。只有我确认后，才给最小 diff。

3.3 Codex CLI / Terminal 使用

- Codex CLI 是运行在 terminal 中的 Codex。它适合在项目根目录中让 agent 读文件、编辑、运行 compile/test 命令。
- 什么时候用：需要跑命令、看真实测试输出、处理多文件项目、在不打开 IDE 时快速检查。
- 在哪个文件夹运行：项目根目录，也就是包含 README、package.json、CMakeLists.txt、pyproject.toml、.git 等文件的位置。
- 读取项目：启动后先问“请总结这个 repo 的结构和运行方式”。
- 运行 compile/test：明确告诉它命令，例如 `g++ main.cpp -o app`、`npm test`、`pytest`。

- 限制修改范围：写“只允许改 `src/input.cpp` 和 `tests/input_test.cpp`，不要改其他文件”。
- 常用命令思路：查看文件结构、运行测试、修复错误、再运行测试、总结 diff。
- 初学者注意：优先使用需要确认的安全模式；不要在个人资料文件夹、下载目录或含隐私文件的目录运行。

请在当前项目中先运行测试并总结结果。不要修改文件。若测试失败，请指出最可能的 3 个原因和下一步验证命令。

3.4 Codex App 桌面版

- Codex App 是桌面端的 agent 工作台，适合同时管理多个项目、多个任务和更长时间的代码工作。
- 和 VS Code/CLI 的区别：VS Code 更贴近编辑器；CLI 更贴近 terminal；Codex App 更像任务控制中心，适合查看 diff、管理 worktree、使用 skills/automations/git 功能。
- worktree：Git 的独立工作目录，同一个 repo 可以开出隔离空间，让不同任务互不干扰。
- diff：修改前后差异。看 diff 是使用 Codex 的核心安全动作。
- 查看修改：按文件查看新增、删除、变更行；确认是否符合 requirement 和你的水平。
- 回退改动：优先用 Git commit/branch 管理；不确定时先问 Codex 解释 diff，再决定 revert 哪个文件或哪一处。
- 适合项目：多文件 assignment、团队项目、需要长时间 debug/测试/PR 的项目。

3.5 Codex 和 GitHub 联动

- 连接 GitHub repository 后，Codex 可以读取 repo、理解 issue/PR、做代码检查或生成 PR 描述。
- 检查 README：让它对照真实项目运行方式，指出 README 中缺失、过时或夸大的地方。
- 修 bug：给 issue、复现步骤、期望行为，让 Codex 在分支上做最小修改并运行测试。
- 创建 pull request：让它总结改动、测试结果、风险和 reviewer 应重点看的文件。
- review pull request：要求它按 severity 排序，重点找 bug、regression、missing tests、rubric 不符。
- 团队项目：用 Codex 做“第二双眼睛”，不要绕过队友 review；每个人仍需理解自己提交的代码。

请 review 这个 PR。请先列出高风险问题，再列出缺失测试，最后给一个简短 summary。不要讨论风格偏好，除非影响可读性或 requirement。

4. Codex 与其他工具的区别

工具	主要用途	是否能读项目	是否能改文件	是否能运行命令	是否适合初学者	是否适合 assignment	风险和限制
Codex	coding agent : 读项目、改文件、运行命令、测试、review	是	是	是	适合，但要学会看 diff	很适合做检查和学习辅助	可能过度修改；需 Git、测试和范围限制
ChatGPT	问答、解释、写草稿、分析片段	通常只能读你上传/粘贴的内容	不能直接改本地文件	通常不能运行你的本地命令	适合概念学习	适合解释和 rubric 分析	上下文不完整时容易猜错
GitHub Copilot	IDE 内代码补全、聊天、局部生成	可理解部分工作区上下文	可在 IDE 中建议修改	通常不负责完整运行流程	适合边写边补全	可辅助写小段代码	容易让学生无意识接受代码
Microsoft Copilot	办公、搜索、企业知识和通用辅助	一般不以代码项目为核心	不直接改 repo	不直接运行项目命令	适合资料整理	适合报告/解释辅助	编程项目上下文弱
Cursor	AI-first IDE，聊天、补全、项目编辑	是	是	可通过终端/工具联动	适合愿意换 IDE 的学生	适合，但同样需规范使用	需管理模型、隐私和自动修改范围
VS Code IntelliSense	语法提示、类型提示、跳转定义	读当前项目符号	不会主动改逻辑	否	非常适合基础开发	适合防低级错误	不能理解复杂需求或 rubric
普通搜索引擎	查资料、文档、错误信息	否	否	否	适合查概念	适合找官方文档	答案分散、质量不一、可能过期
Stack Overflow	具体问题的社区答案	否	否	否	适合查典型 bug	适合作为参考	代码片段可能不适配你的项目或课程

5. 初学者快速上手路线

天数	学习目标	操作步骤	推荐 prompt	练习任务	常见错误
Day 1：认识 Codex	知道 Codex 是 coding agent，不是代写工具。	阅读本文第 1-4 节；打开一个小项目；让 Codex 只总结项目结构。	请用初学者语言解释 Codex 能在这个项目中做什么，先不要改文件。	练习：找出项目入口文件和运行命令。	错误：一上来就让它写完整 assignment。

天数	学习目标	操作步骤	推荐 prompt	练习任务	常见错误
Day 2 : 用 Codex 解释代码	能看懂函数、变量和流程。	选择 50-100 行代码；让它按步骤解释；自己画流程图。	请按输入、处理、输出解释这个函数，并指出我需要复习的语法。	练习：自己改一个变量名或输出格式。	错误：只读解释，不自己运行。
Day 3 : 用 Codex debug	会提供预期/实际/复现步骤。	运行程序；记录错误；让 Codex 先分析可能原因。	这是预期输出和实际输出。请先提出假设和验证方法，不要直接改。	练习：修一个小 bug 并复测。	错误：只贴一句“it does not work”。
Day 4 : 用 Codex 检查 assignment	会用 requirement 和 rubric 做清单。	粘贴 requirement、rubric、自己的文件结构；要求逐项检查。	请把 rubric 转成 checklist，并标注我的代码证据在哪个文件。	练习：找 3 个未满足项。	错误：没给 rubric 就问能不能拿高分。
Day 5 : 用 Codex 学 VS Code 联动	会限制当前文件/多个文件范围。	在 VS Code 打开项目；让 Codex 解释当前文件；再让它追踪一个函数调用。	请只检查当前文件，不要修改。然后告诉我哪些其他文件会影响它。	练习：看一次 diff。	错误：一次允许它改变整个项目。
Day 6 : 用 Codex 学 Git/GitHub	理解 commit、branch、PR 和 diff。	初始化或打开 Git repo；做一次小修改；让 Codex 解释 diff。	请解释这个 git diff 每一处修改的目的，以及是否有风险。	练习：创建分支并提交。	错误：不 commit 就大量修改。
Day 7 : 用 Codex 完成一个小项目检查	能完成提交前检查流程。	提供 requirement；运行测试；检查 README；准备老师问题。	请按最终提交 checklist 检查项目：功能、测试、README、代码风格、rubric、风险。	练习：生成最终自查报告。	错误：不理解代码就提交。

6. 学生写 assignment 时如何正确使用 Codex

这一节最重要：Codex 应该帮助你理解要求、发现问题、学习修复方法，而不是替你完成全部核心工作。课程作业的最终责任永远在你。

正确提供材料

你要提供什么	为什么重要	示例
assignment requirement	Codex 需要知道必须实现什么、输入输出是什么、禁止什么。	PDF/截图/文字均可，最好包含完整任务描述。

你要提供什么	为什么重要	示例
rubric	rubric 决定评分重点，例如 correctness、style、testing、documentation。	把每个评分项贴出来，让 Codex 转成 checklist。
自己的代码	Codex 应该基于你的现有思路做检查，而不是另起炉灶。	提供相关文件或项目目录。
课程范围	避免生成超出课程内容的高级库、架构或语法。	例如：只能用 loops、functions、arrays/vector，不能用 templates 或 external libraries。
你想要的帮助类型	分析、debug、解释、最小修改、测试用例是不同任务。	写清楚“先分析，不要改”。

推荐 workflow

1. 上传/提供 assignment 要求：让 Codex 用中文总结任务目标、输入输出、限制和提交物。
2. 提供 rubric：让 Codex 转成 checklist，不要马上写代码。
3. 提供自己的代码：说明哪些部分是你自己写的、哪些还没完成。
4. 让 Codex 检查是否满足要求：逐项标注证据文件和缺失项。
5. 让 Codex 找 bug：提供运行结果、错误输出和复现步骤。
6. 让 Codex 提供最小修改：限定文件和范围，不要重写整体结构。
7. 自己理解并手动修改：尤其是 assignment 核心逻辑，确保能解释。
8. 让 Codex 生成解释和答辩问题：练习讲清为什么这样写。
9. 最终检查：运行测试、看 diff、检查 README、确认没有隐私文件或临时文件。

避免 academic misconduct 的原则

- 不要提交自己不理解、不能解释的代码。
- 不要让 Codex 直接完成整份作业的核心算法并原样提交。
- 遵守学校关于 AI 工具使用和引用的政策；不确定时间 tutor/lecturer。
- 保留学习痕迹：自己的草稿、笔记、测试、修改原因。
- 使用 Codex 做解释、检查、提示、debug 和反馈，比“帮我写完”更安全、更有学习价值。

我会提供 assignment requirement、rubric 和我已经写好的代码。请你先只做检查：1) requirement checklist；2) rubric 对照；3) bug 风险；4) 哪些地方可能超出课程范围。不要直接给完整答案。

7. 如何写高质量 Codex Prompt

好的 Codex prompt 通常包含：背景、目标、输入材料、限制范围、输出格式、是否允许修改、学习要求。

Prompt 部分	你应该写什么	例子
背景	课程、语言、水平、环境。	我是 C++ 初学者，只学过 function、loop、array/vector。
目标	你要它解释、debug、检查还是修改。	请检查 menu system 的 input validation。
范围	允许看/改哪些文件。	只允许修改 main.cpp，不要改 README。
限制	禁止高级技术、禁止重写、保持风格。	不要使用 class、template、lambda 或外部库。
流程	先分析、再询问、最后给最小 diff。	先列问题和方案，等我确认后再改。
输出格式	表格、清单、diff summary、步骤。	输出：问题、原因、最小修改、测试方法。

7.1 解释代码 prompt

请用初学者能懂的中文解释这段代码。按：整体目的、每个函数、关键变量、执行流程、可能出错点、我应该复习的语法点来讲。

请不要改代码。请逐行解释这个函数，并最后用一个具体输入例子模拟程序如何运行。

7.2 Debug prompt

这是预期结果、实际结果和相关代码。请先列出最可能的 3 个原因，以及每个原因如何验证。不要马上修改。

请帮我定位 bug。要求：只找原因和给最小修复建议，不要重写整个程序。

7.3 Compile error prompt

请解释这个 compile error 的含义，指出真正出错的源代码位置，并给 C++ 初学者能理解的最小修复。

请把下面的编译错误按优先级排序：哪些是根本错误，哪些可能是连锁错误？

7.4 Assignment rubric 检查 prompt

请把这个 rubric 转成 checklist，并用我的代码逐项标注：已满足/部分满足/未满足/需要确认。

请只检查是否满足 assignment requirement，不要直接帮我写缺失功能。每个问题给证据位置。

7.5 C++ 初学者 prompt

请保持 C++ beginner style，只使用我学过的 if/else、loops、functions、arrays/vector、basic file I/O。不要用高级库或 OOP 重构。

请检查我的 C++ input validation。只给最小修改，并解释 cin fail state、clear 和 ignore 的作用。

7.6 Python prompt

请解释这个 Python script 的数据流，并指出哪些变量名、函数拆分或异常处理可以更清楚。

请帮我 debug 这个 Python error。先解释 traceback 从下往上怎么看，再给最小修复。

7.7 HTML/CSS/JavaScript prompt

请检查这个网页 assignment 的 HTML/CSS/JS：布局、responsive、accessibility、交互 bug。先列问题，不要改。

请只改 CSS，让页面在 mobile 和 desktop 都不重叠。不要改 HTML 结构，除非必须。

7.8 Cybersecurity lab prompt

这是授权 cybersecurity lab 的 terminal output。请解释每条命令和输出代表什么，并帮我写防御性 report，不要提供非法攻击步骤。

请分析这个 vulnerability 的原理、影响、修复建议和报告写法。避免提供真实目标利用流程。

7.9 GitHub repo review prompt

请阅读这个 repo 并总结：项目目的、运行方式、主要文件、测试方式、潜在风险。先不要修改。

请检查 README 是否和实际项目一致，列出缺失的安装、运行、测试和限制说明。

7.10 Pull request review prompt

请 review 这个 PR，优先找 bug、regression、missing tests 和 requirement 不符。按 severity 排序。

请生成 PR review comments，要求具体、礼貌、可执行，不要纠结无关风格。

7.11 README 生成 prompt

请根据项目内容生成 beginner-friendly README：简介、功能、安装、运行、测试、文件结构、已知限制。

请更新 README，但不要夸大项目功能。所有命令必须来自实际项目配置。

7.12 老师提问准备 prompt

请根据我的代码生成 10 个老师可能问的问题，并给简短参考回答，重点包括边界情况和设计选择。

请扮演 tutor，对我的代码进行口头检查。每次问一个问题，等我回答后再指出不足。

7.13 代码简化 prompt

请找出这段代码中可以简化的地方，但保持初学者风格，不要使用高级语法。

请提出 3 个 readability 改进建议，每个建议说明是否值得改、风险是什么。

7.14 不要太像 AI 的 prompt

请保持我的原始 coding style，只做必要修复。不要把代码改得过于高级或企业化。

请解释哪些修改可能看起来不像我当前水平，并提供更 beginner-friendly 的替代写法。

7.15 只做最小修改 prompt

请只做让测试通过的最小修改。不要重命名大量变量，不要重构文件结构，不要引入新库。

请先输出最小修改计划和影响文件列表；我确认后再给 diff。

8. Codex 安全使用指南

- 不能让 Codex 随便改整个项目：它可能误解 requirement、改到无关文件、引入新 bug。
- 先备份：最简单的备份方式是 Git commit；也可以复制一份项目文件夹。
- 建议使用 Git：Git 能记录修改历史，让你看 diff、撤销错误、创建 branch。
- commit：一次有意义的保存点，例如“fix input validation”。
- branch：独立开发线，适合让 Codex 在新分支上尝试修改。
- diff：修改前后的差异，是接受 Codex 修改前必须看的东西。
- rollback/revert：回退不想要的修改。初学者应先问清楚 diff，再回退。
- 检查修改文件：使用 Git diff、source control 面板或 Codex App 的 diff 视图。
- 确认没有破坏原功能：运行原有测试、手动测试核心流程、检查 edge cases。
- 避免泄露隐私：不要在包含护照、银行卡、私人照片、密码、API key 的文件夹运行。
- 限制查看范围：只在项目根目录运行，并明确“只看/只改这些文件”。
- 避免危险命令：不要让它执行删除大量文件、上传隐私、改系统配置、操作真实攻击目标的命令。

安全检查清单

检查项	完成
我在正确的项目根目录运行 Codex	☐
项目已经有 Git commit 或备份	☐
prompt 写清楚允许修改的文件范围	☐

检查项	完成
没有把隐私文件、密码、API key 放进项目	☐
我会先看 diff，再接受修改	☐
我运行了 compile/test/manual test	☐
我能解释每一处关键修改	☐
我确认没有违反学校 AI/academic integrity 政策	☐

9. Codex 和 Git / GitHub 基础

术语	初学者解释
Git	本地版本控制工具，记录文件修改历史。
GitHub	在线托管 Git repository 的平台，适合备份、协作和 PR。
repository/repo	一个被 Git 管理的项目文件夹。
commit	一次保存点，包含修改内容和说明。
branch	独立开发线，适合尝试功能或修 bug。
pull request/PR	请求把一个分支的修改合并到另一个分支，并让别人 review。
merge	把分支修改合并进目标分支。
conflict	两边改了同一位置，Git 不知道该保留哪个，需要人工解决。
README	项目说明书，告诉别人项目是什么、如何运行、如何测试。
.gitignore	告诉 Git 哪些文件不要追踪，例如 build output、env 文件、临时文件。

Codex 和 GitHub 联动有用，是因为它能基于真实 diff、issue、PR 和测试结果做检查。它可以帮你学习 Git 命令含义、解释 conflict、生成 PR 描述，但最终 merge 前仍要人工 review。

我对 Git 不熟。请解释当前 git status 和 git diff，告诉我哪些文件被修改、每个修改可能来自什么任务，以及下一步应该 commit、继续改还是回退。

10. Codex 在不同课程中的使用方法

10.1 C++ programming assignment

- 检查 function 是否职责清楚、参数和返回值合理。

- 检查 input validation，例如非数字输入、范围、空输入。
- 检查 menu system 是否能循环、退出、处理 invalid choice。
- 检查 file input/output 是否关闭文件、处理文件不存在。
- 检查 array/vector 使用是否越界、是否符合 beginner-level code style。

请检查我的 C++ assignment。要求只使用 beginner-level C++，重点检查 functions、input validation、menu、file I/O、array/vector 和 edge cases。不要重写整体结构。

10.2 Python assignment

- 解释脚本整体流程和每个函数职责。
- debug traceback、类型错误、文件路径错误。
- 检查数据处理流程，例如读取 CSV、清理数据、统计结果。
- 建议函数拆分，让 main flow 更清楚。

请检查这个 Python assignment 的数据处理流程。先解释每个函数，再指出 bug 风险和可以拆分的地方，保持初学者风格。

10.3 Web development assignment

- 检查 HTML 语义结构、CSS layout、JavaScript interaction。
- 检查 responsive design 是否在 mobile/desktop 都可用。
- 检查 accessibility：label、alt text、keyboard focus、contrast。
- 提出 UI improvement，但不做夸张 redesign。

请检查我的 HTML/CSS/JS assignment：layout、responsive、accessibility 和 JS interaction。先列问题和优先级，不要直接改。

10.4 Cybersecurity lab

- 解释命令和 terminal output。
- 分析 vulnerability 的原理、影响和修复建议。
- 帮助把实验观察写成 report explanation。
- 拒绝帮助未经授权攻击、真实目标利用、凭证窃取、隐蔽持久化。

这是授权 lab 的输出。请解释每一步发生了什么、它证明了什么安全概念、如何写进报告，并给防御/修复建议。

10.5 Systems analysis / documentation

- 生成 README、system overview、technical explanation。
- 把项目功能整理成 user guide。
- 帮助写 project reflection：学到什么、限制是什么、未来改进。

- 把复杂技术语言改成老师/同学能懂的说明。

请根据项目生成 documentation：README、system overview、user guide 和 reflection outline。内容必须和真实功能一致，不要夸大。

11. Codex 的限制

- 可能理解错 requirement，尤其是老师课堂口头补充或 rubric 细节。
- 可能生成错误代码，即使解释听起来很自信。
- 可能过度修改，把小 bug 变成大重构。
- 可能使用超出课程范围的技术，让代码不像学生自己写的。
- 可能忽略 edge cases，例如空输入、重复数据、文件不存在、网络失败。
- 可能不能理解老师的具体评分偏好。
- 运行环境可能和你的电脑不同，例如路径、版本、依赖、操作系统。
- 你必须自己检查、运行、理解、解释，并遵守课程政策。

12. 常见错误和解决方法

常见错误	为什么是问题	正确做法	改进 prompt 示例
prompt 太短	Codex 缺少背景会猜测。	补充语言、目标、错误、范围。	我是 Python 初学者。请根据 traceback 和代码定位 bug，先不要改。
没给 assignment requirement	它不知道真正要满足什么。	先贴 requirement，让它总结。	请先总结 requirement 和限制，再问我要哪些代码文件。
没给 rubric	可能忽略评分重点。	把 rubric 转 checklist。	请按 rubric 逐项检查，并标注证据文件。
直接让它写完整代码	学习价值低，且可能违反政策。	让它给思路、伪代码、提示。	请给实现步骤和关键提示，不要写完整答案。
不看 diff 就接受修改	可能引入无关改动。	逐文件检查 diff。	请解释这个 diff 每一处为什么需要。
不理解代码就提交	答辩/检查时无法解释。	要求生成问题并自己回答。	请根据代码问我 10 个老师可能问的问题。
在错误文件夹运行	可能看不到项目或看到隐私文件。	在项目根目录运行。	请先确认当前目录是否像项目根目录，并说明依据。
没有备份	改坏后难恢复。	先 git commit 或复制备份。	请检查 git status，告诉我是否适合开始修改。
让它使用太高级的库	可能超出课程范围。	明确课程知识范围。	只能使用我们学过的基础语法，不要引入外部库。

常见错误	为什么是问题	正确做法	改进 prompt 示例
没有测试修改后的代码	修复可能破坏其他功能。	运行 compile/test/manual test。	请给我修改后的测试清单和运行命令。

13. Codex 学习路线图

阶段	学习目标	练习任务	推荐 prompt
Beginner level	会让 Codex 解释代码、找简单 bug、检查 assignment。	每天选一个函数解释；用 rubric 做 checklist；修一个小 bug。	请解释这段代码，并给我 3 个检查理解的问题。
Intermediate level	会使用 VS Code 联动、读取整个项目、看 diff、用 Git 保存版本。	打开一个 repo；让 Codex 总结结构；改一处 bug；看 diff；commit。	请阅读项目结构，说明入口、主要模块、测试命令和修改风险。
Advanced level	会使用 CLI、GitHub PR、code review、生成测试、改进项目结构。	在分支上让 Codex 修 bug 并跑测试；生成 PR 描述；review 自己 PR。	请对这个 PR 做高信号 code review，优先找 bug 和 missing tests。

14. Codex Prompt Library

学习类

1. 请像大学 IT tutor 一样教我这个概念。先用生活化比喻，再给编程例子，最后给 3 个练习题。
2. 请根据我的代码列出我需要复习的语法点，并按优先级排序。
3. 请不要直接给答案。请用 Socratic tutoring 的方式一步步提示我。

解释代码类

4. 请解释这个文件的作用、入口函数、主要数据结构和函数调用关系。
5. 请用一个具体输入例子 trace 这段代码的执行过程。
6. 请指出这段代码中最容易让初学者误解的 5 个点。

debug 类

7. 请根据错误输出和代码定位 bug。先列假设、验证方法、最小修复。

8. 请帮我构造一个最小复现 case，确认这个 bug 是否真的存在。
9. 请比较预期输出和实际输出，指出逻辑在哪里偏离。

assignment 检查类

10. 请把 requirement 转成 checklist，并对照我的代码逐项检查。
11. 请检查我的代码是否超出课程范围，并给 beginner-friendly 替代写法。
12. 请找出最影响分数的 5 个问题，按 rubric 权重排序。

GitHub 类

13. 请总结这个 repo：目的、运行方式、主要文件、测试方式和已知风险。
14. 请帮我写 PR description：改了什么、为什么改、如何测试、风险。
15. 请检查 .gitignore 是否遗漏了 build、env、cache 或 IDE 文件。

VS Code 类

16. 请只检查当前文件，输出 bug 风险和 readability 建议，不要修改。
17. 请追踪这个函数在哪些文件被调用，并解释调用链。
18. 请解释我应该如何如何在 VS Code 中查看这些修改的 diff。

Terminal 类

19. 请解释这条 terminal command 的每个参数，以及运行前有什么风险。
20. 请根据这段 terminal output 判断下一步该运行什么命令。
21. 请给我一组安全的 compile/test 命令，不要删除或移动文件。

cybersecurity lab 类

22. 请解释授权 lab 中这段扫描输出，说明发现、风险、证据和修复建议。
23. 请帮我把 vulnerability analysis 写成 report 段落，避免提供非法利用步骤。
24. 请区分这个 lab 中哪些行为只适用于授权环境，哪些是日常防御建议。

report/documentation 类

25. 请根据项目生成 README，包括 setup、run、test、features、limitations。

- 26. 请把我的技术说明改成适合 assignment report 的中文，保留关键英文术语。
- 27. 请生成 project reflection：我做了什么、遇到什么问题、如何解决、未来改进。

老师提问准备类

- 28. 请扮演老师，根据我的代码问 10 个 viva questions，并给参考回答。
- 29. 请找出我最可能解释不清楚的 5 个代码点，并教我如何回答。
- 30. 请根据 rubric 模拟 final demo 检查流程。

最小修改类

- 31. 请只做最小修改，保持我原有结构和变量命名，不要引入新库。
- 32. 请先给修改计划和影响文件列表，等我确认后再改。
- 33. 请把每处修改解释成：问题、原因、修改、如何测试。

15. 最后总结

Codex 最重要的价值，不是让学生少学代码，而是让学生更快进入真实编程循环：理解需求、阅读代码、定位问题、做小修改、运行测试、看 diff、解释自己的选择。

主题	一页总结
最重要的价值	把“我不知道从哪里开始”变成“我知道下一步该检查什么”。它能把项目、错误、测试、Git 和文档连接起来。
学生最应该怎么用	用它解释代码、生成检查清单、定位 bug、设计测试、准备老师提问。关键是你自己要自己理解和验证。
哪些事情不要做	不要让它直接完成整份作业；不要提交不理解的代码；不要不看 diff；不要在有隐私资料的文件夹运行；不要用于未授权攻击。
最推荐的上手路线	先用 ChatGPT/Codex 解释代码，再在 VS Code 中检查当前文件，然后学习 Git diff/commit，最后尝试 CLI 或 Codex App 处理完整项目。
你的下一步	拿一个小 assignment 或练习项目，用本文 Day 1-7 路线走一遍。每次只做一个小目标：解释、debug、检查、测试、总结。

最终原则：Codex 可以帮你更快学习，但不能替你承担理解、判断、测试和诚信责任。最稳的做法是：先让它

分析，再让它给最小建议，最后由你亲自理解、修改、运行和提交。

参考资料与资料日期

资料整理日期：2026-05-09。Codex 的界面、模型、权限、额度和集成方式可能更新，实际使用时请以官方页面和你本机版本为准。

- OpenAI Help Center: Using Codex with your ChatGPT plan, <https://help.openai.com/en/articles/11369540-using-codex-with-your-chatgpt-plan/>
- OpenAI Help Center: OpenAI Codex CLI - Getting Started, <https://help.openai.com/en/articles/11096431-openai-codex-cli-getting-tarted>
- OpenAI Codex product page, <https://openai.com/codex/>
- OpenAI Developers: Codex use cases, <https://developers.openai.com/codex/explore/>
- OpenAI Codex GitHub repository, <https://github.com/openai/codex>